[SUIHUB LAGOS / DAY 03]

Real-Time Sui

GraphQL Subscriptions and the Data Stack After JSON-RPC

MAP THE STACK / BUILD TILL / MIGRATE YOUR APP

SUIHUB LAGOS / ACCOUNT LAYER WEEK

Why this session, now?

JULY 2026

JSON-RPC shutoff

Foundation mainnet disablement began the week of 27 July.

21 SEPTEMBER

Subscriptions live

GraphQL subscriptions announced for mainnet and testnet.

YOUR APP

Choose its data path.

Read current state, stream activity, retrieve history and migrate.

Four layers. Choose for your workload.

gRPC / WALLET + GAME

Current state and actions

Balances, execution and live checkpoint streams.

GRAPHQL / APP + SHOP

Filtered reads and feeds

Select fields, join history and subscribe to activity.

ARCHIVE / OLD RECEIPTS

History past retention

Retrieve records a full node has pruned.

CUSTOM INDEXER / ANALYTICS

Your own data model

Backfill checkpoints, transform data and own aggregates.

Subscribe to what your screen needs.

TRANSACTIONS

Address, object or call

affectedAddress · affectedObject
function · kind · sentAddress
Checkpoint bounds + cursor

EVENTS

Sender, module or type

sender · module · type
Checkpoint bounds + cursor
Select only the fields you need.

Till selects digest, sender, status, timestamp, checkpoint and balance changes.

The actual Till subscription.

```
subscription Till($shop: SuiAddress!, $after: String, $afterCheckpoint: UInt53) {
 transactions(
   filter: { affectedAddress: $shop, afterCheckpoint: $afterCheckpoint }, after: $after
 ) {
   cursor
   node {
     digest sender { address }
     effects {
       status timestamp checkpoint { sequenceNumber }
       balanceChanges(first: 50) { nodes { owner { address } coinType { repr } amount } }
     }
   }
 }
}
```

A matching transaction becomes a receipt only after success and balance-change checks.

POST + SSE. Stream with fetch.

```
const res = await fetch(url, {
   method: 'POST',
   headers: { 'content-type': 'application/json', accept: 'text/event-stream' },
   body: JSON.stringify({ query, variables }), signal,
 });
 if (!res.ok || !res.body) throw new Error(`Subscription HTTP ${res.status}`);
 if (!res.headers.get('content-type')?.includes('text/event-stream')) {
   throw new Error('Expected an event stream');
 }
 const reader = res.body.pipeThrough(new TextDecoderStream()).getReader();
```

POST /graphql/subscriptions / text/event-stream / EventSource cannot POST the query

Resume from a durable position.

01

First start

afterCheckpoint sets the opening boundary

02

Every delivery

Persist the cursor with processed data

03

Reconnect

after resumes from the last saved cursor

04

Catch up

Replay missed activity, then switch to live

Checkpoint order → transaction order → event order. Respect subscriber throttling.

A small indexer: commit data and cursor.

```
db.exec('BEGIN');
try {
 for (const p of payments) {
   insert.run(p.digest, p.coinType, p.amount,
     p.sender, p.checkpoint, p.timestamp, p.cursor);
 }
 saveCursor.run(shop, d.transactions.cursor,
   new Date().toISOString());
 db.exec('COMMIT');
} catch (e) {
 db.exec('ROLLBACK');
 throw e;
}
```

pnpm indexer → stop → pay → restart → pnpm totals / Atomic writes + unique receipt keys

When should you use the Rust framework?

SQLITE / POSTGRES WRITER

A focused application feed

One shop, selected activity,
a small derived table
and straightforward totals.

RUST INDEXING FRAMEWORK

A checkpoint pipeline

Large backfills, parallel work,
custom schemas, complex joins
and controlled reprocessing.

Find history beyond the full node.



DATE

7 October 2025

CHECKPOINT

250,000,000

RESULT

GraphQL found the receipt.

Known receipt: 7 October 2025 / Checkpoint 250,000,000 / GraphQL found it

Public endpoints are for development.

QUERY BUDGET / OBSERVED

5,000 bytes · 40 seconds

Query payload and timeout limits vary by service.

PAGINATION / OBSERVED

20 default · 50 maximum

Page Query.transactions and nested collections.

BEFORE LAUNCH

Choose a production endpoint.

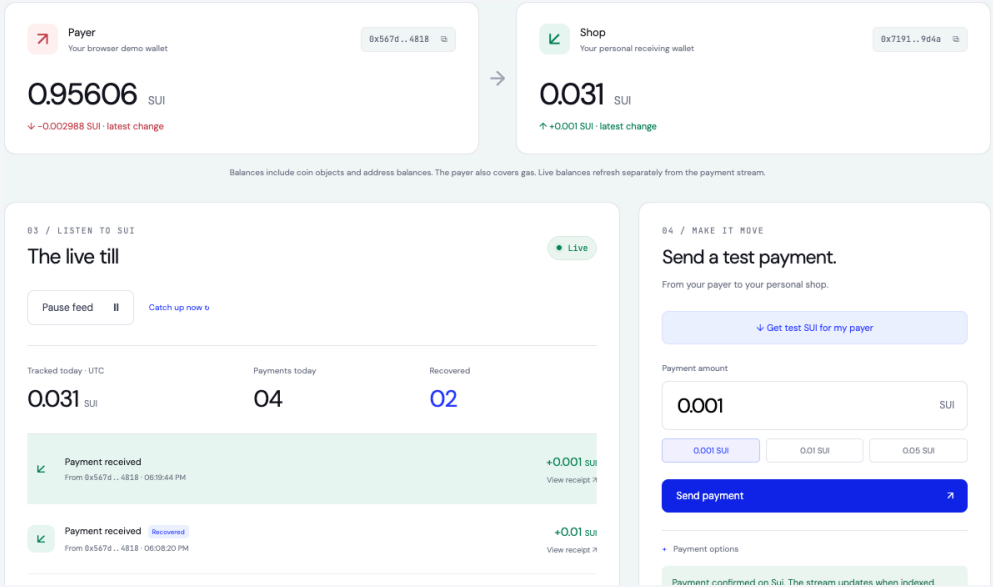
Check quotas, throttling, retention, auth and uptime.

REAL DEMO GOTCHA

Confirmed, but missing live?

Address-balance deposits appeared after catch-up.

Build Till on the stack we just mapped.



PERSONAL

Your payer. Your shop.

LIVE

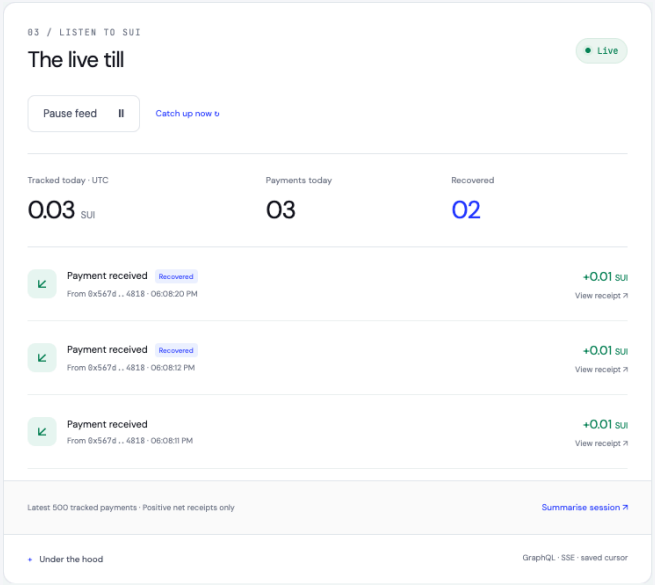
A receipt from the stream.

RECOVERABLE

A saved place to resume.

Current verified walkthrough: test SUI. The advertised USDC payment still needs verification.

Pause mid-sale. Recover. Reconcile.



01 / PAUSE

The listener stops.

02 / PAY

The chain keeps going.

03 / RESUME

Missed receipts return.

Reload → resume / Switch payer → personal shop / Reset tracking → funds stay onchain

Migrate a JSON-RPC call.

```
pnpm migrate:legacy   # inspect the JSON-RPC error body
pnpm migrate:grpc     # current balance through gRPC
pnpm migrate:graphql  # structured indexed query

// From migration/migrated-grpc.ts:
const { balance } = await client.getBalance({ owner: SHOP });
console.log('getBalance     ', balance);
```

suix_getBalance → getBalance or Address.balance / Transaction and event examples included

Now migrate one call in your own app.

01

Find

Locate one JSON-RPC read or polling loop

02

Choose

gRPC, GraphQL, subscription or indexer

03

Replace

Port fields, response shape and pagination

04

Prove

Check results, reconnect and old-history behaviour

Deliverable: one working migrated call, with a clear reason for the chosen data service.

day3-till.vercel.app

github.com/Nuel-osas/day3-till